ERRUM ERP INTERNAL GUIDEBOOK

Cash Sheet Consolidated Process Guide

A code-grounded, finance-friendly manual for the new single cash-sheet system. It explains every update path, transaction source, cancellation/refund behavior, return-exchange impact, formula, example, and QA test needed to stop repeated cash-sheet bugs.

Based on	Latest uploaded Errum FE/BE ZIPs + previous cash-sheet guide
Main implementation change	cash-sheet-new logic has been merged into canonical /cash-sheet. Backend /cash-sheet-new routes removed.
Frontend focus	app/cash-sheet/page.tsx, services/cashSheetService.ts, cash-sheet panels, Sidebar, access map
Backend focus	CashSheetController.php and routes/api.php
Purpose	Explain exactly how cash sheet tracks sales, cash, bank, costs, admin movements, online/COD/SSLZC/Pathao, owners, cancellation, refund, and exchange.

Prepared for developers, finance/admin users, branch users, and QA testers. Designed to be read like a practical operations book, not just API notes.

How to use this guide

Read this as the permanent reference for the consolidated cash sheet. The old cash-sheet-new route was only a development/parallel page. The cash sheet is now one page, one route, one backend calculator, and one mental model.

Table of contents

1. Executive summary: what changed
2. Code map: frontend, backend, and removed duplicate route
3. The mental model: live aggregation, not a saved spreadsheet
4. Frontend behavior and visible columns
5. Backend monthly pipeline
6. Business date rules and the yesterday/today fix
7. Branch sales and cash/bank payments
8. Cancellations and refunds - new corrected behavior
9. Return-exchange and Ex/On tracking
10. Daily costs from cash-sheet panel and accounting expenses
11. Admin entries: salary, cash-to-bank, SSLZC, Pathao
12. Online/ecommerce: advance, SSLZC visibility, COD/Due, disbursement
13. Owner entries and final after-cost totals
14. Full real-value day simulation
15. Scenario-by-scenario simulations
16. QA test plan and SQL/debug checklist
17. Developer handoff checklist

1. Executive summary: what changed

The system now treats /cash-sheet as the only real cash sheet. The previous /cash-sheet-new logic has been copied into the canonical page and service. Backend /api/cash-sheet-new routes have been removed to stop duplicate-route confusion. The frontend /cash-sheet-new page is only a redirect to /cash-sheet for old bookmarks.

Problem from previous guide	Implemented fix	Why this matters
cash-sheet-new route defined twice	Removed backend /cash-sheet-new route groups. Only /api/cash-sheet remains.	Developers and QA now test one endpoint, not two aliases.
Entry panels used /cash-sheet while new grid used /cash-sheet-new	All official cash-sheet UI and services now use /cash-sheet.	Branch cost, admin, owner, and monthly grid reconcile through the same route.
Payment selected for yesterday could hydrate today	Payment date expression now prefers op.payment_received_date before completed_at.	Backdated POS/offline payments land on the intended business date.
Completed payments disappeared after order cancellation/refund	Payment loaders now count completed cash-flow even when parent order status later changes. Refund rows subtract separately.	Cancellation/refund dates show the real money-in and money-out trail.
displayed_cash clamped to zero	Branch cash can now show negative when salary/cost/bank-transfer exceeds raw cash.	Cash shortage is visible instead of silently hidden.
Inactive branches vanished from old months	Cash sheet loads active stores plus inactive stores with month activity.	Historical sheets do not lose branch columns.
COD/Due bucket too broad	Outstanding online due is limited to COD/courier/COD-metadata orders.	Non-COD partial dues do not inflate COD/Due.
Refund method null was ignored	Null refund method now counts as money refund instead of being dropped.	Older refund rows still affect cash/bank.

Main rule: Sales columns are commercial order value. Cash/bank columns are real money movement. This distinction is the heart of the corrected cancellation/refund logic.

2. Code map

2.1 Frontend files

File	Role after consolidation
app/cash-sheet/page.tsx	Canonical monthly grid. It now contains the former cash-sheet-new features: Dhaka month handling, COD/Due label, online buckets, branch role filtering, negative value display, and monthly footer.
app/cash-sheet-new/page.tsx	Deprecated redirect only. It sends old bookmarks to /cash-sheet.
services/cashSheetService.ts	Canonical API wrapper for /cash-sheet endpoints. Types now include cod_due, cod_collected, and refunds fields.
services/cashSheetNewService.ts	Compatibility shim that re-exports cashSheetService. It should not be used for new development.
app/cash-sheet/branch-cost/page.tsx	Manual branch daily cost panel. Writes branch_cost_entries and mirrored completed expense/payment.
app/cash-sheet/admin/page.tsx	Admin panel for salary set-aside, cash-to-bank, SSLZC received, and Pathao received.
app/cash-sheet/owner/page.tsx	Owner investment/cost panel.
components/Sidebar.tsx	Cash Sheet menu now points to the single monthly /cash-sheet page.
lib/accessMap.ts	cash-sheet allows super-admin, admin, branch-manager, and pos-salesman. Branch roles still only see own store in the page.

2.2 Backend files

File	Role after consolidation
routes/api.php	Only /api/cash-sheet is registered for cash sheet. /api/cash-sheet-new groups have been removed.
app/Http/Controllers/CashSheetController.php	Single monthly calculator and write endpoint controller for sheet, entries, branch costs, admin entries, and owner entries.
BranchCostEntry, AdminEntry, OwnerEntry models	Manual cash-sheet entry tables.
OrderPaymentController / PaymentController	Create payment rows and payment_received_date values that the cash sheet later reads.
RefundController / ProductReturnController / ExchangeController	Create completed refunds and exchange payment/refund rows that cash sheet reads as money movement.
ExpenseController + ExpensePaymentObserver	External expenses and branch-cost mirror ledger generation.

2.3 Route map

Method	Route	Controller method	Purpose
GET	/api/cash-sheet?month=YYYY-MM	index	Builds full monthly sheet.
GET	/api/cash-sheet/entries?date=YYYY-MM-DD	entries	Loads manual/admin/owner/accounting entries for one date.
POST	/api/cash-sheet/branch-cost	storeBranchCost	Adds branch daily cost.
DELETE	/api/cash-sheet/branch-cost/{id}	destroyBranchCost	Deletes branch cost and cancels linked mirror expense.
POST	/api/cash-sheet/admin	storeAdmin	Adds salary, cash-to-bank, SSLZC, Pathao.

Method	Route	Controller method	Purpose
DELETE	/api/cash-sheet/admin/{id}	destroyAdmin	Deletes admin entry and cancels ledger pair.
POST	/api/cash-sheet/owner	storeOwner	Adds owner investment/cost.
DELETE	/api/cash-sheet/owner/{id}	destroyOwner	Deletes owner entry and cancels ledger pair.

3. Mental model: live aggregation, not a saved spreadsheet

The cash sheet still does not save one daily-row record. It is a live monthly report rebuilt from source tables every time the page loads. The page shows one row per date, but that row is produced from sales, payments, splits, refunds, branch costs, expense payments, admin entries, online buckets, disbursements, and owner entries.

```
GET /api/cash-sheet?month=2026-07
  -> calculate dateFrom/dateTo for the month
  -> load active stores + inactive stores with activity in the month
  -> load branch sales by order date
  -> load branch cash/bank by payment/refund dates
  -> load Ex/On exchange movements
  -> load branch costs from manual and accounting expense sources
  -> load admin entries for salary, cash-to-bank, SSLZC, Pathao
  -> load online sales, advance, SSLZC-visible payments, COD/Due, online refunds
  -> load owner entries
  -> calculate daily rows and monthly summary
```

Update style	What happens	Examples
Direct cash-sheet entry	User posts to a cash-sheet entry table. The monthly grid changes after reload because controller reads that entry table.	Branch Daily Cost, Admin Panel, Owner Panel
Indirect business event	Another module changes a source table. No cash-sheet row is written. The monthly grid changes because aggregation sees new source data.	POS payment, split payment, refund, online order, expense payment, exchange top-up
Status update / cancellation	Order status may remove the order from sales, but completed payments/refunds remain as cash-flow events.	Cancel paid social order; refund branch POS order

Debugging mindset: when one number is wrong, identify the visible column first. Then go directly to that column's source tables. Do not search for a saved cash_sheet_rows table.

4. Frontend behavior and visible columns

The monthly page is read-only. Users add operational entries through the Branch Costs, Admin Panel, and Owner Panel pages. Admin/super-admin see all branches and online/owner sections. Branch users are authorized, but the page filters visibleStores to their own store ID.

4.1 Page load sequence

• Calculate current month using Asia/Dhaka date.

• Check role. Admins see all stores; branch roles see only their own store.

• Call cashSheetService.getSheet(month), which sends GET /cash-sheet with month and cache-busting timestamp.

• Store response.stores, response.data, and response.summary in React state.

• Render a wide monthly table: date, every branch, online/ecommerce, disbursement, day totals, owner totals.

• Display negative values in red instead of printing 0. This is important for branch cash deficit.

4.2 Column map

UI section	Column	Backend field	Meaning
Branch	Sale	branches[].daily_sale	Commercial sale value for branch order types on order date.
Branch	Cash	branches[].cash	Raw cash payments minus cash refunds, salary, cash-paid costs, and cash-to-bank. Can be negative now.
Branch	Bank	branches[].bank	Raw non-cash payments minus bank-paid costs plus cash-to-bank.
Branch	Ex/On	branches[].ex_on	Exchange surplus collected minus exchange refund paid.
Branch	Salary	branches[].salary	salary_setaside admin entries.
Branch	Cost	branches[].daily_cost	Manual branch costs + completed external expense payments.
Branch	->Bank	branches[].cash_to_bank	Admin cash-to-bank movement.
Online	Sales	online.daily_sales	Social/ecommerce order value on order date.
Online	Advance	online.advance	Social-commerce non-COD received amount, added to day bank.
Online	SSLZC	online.online_payment	Ecommerce non-COD gateway payments visible as receivable; not final bank until SSLZC received.
Online	COD/Due	online.cod	COD/courier outstanding due + delivered COD collections - COD refunds.
Disbursements	SSLZC Received	disbursements.sslzc_received	Admin settlement received, added to final bank.
Disbursements	Pathao Received	disbursements.pathao_received	Admin courier/COD settlement received, added to final bank.
Day totals	Cash	totals.cash	Sum of branch displayed cash.
Day totals	Bank	totals.bank	Sum branch bank + online advance.
Day totals	Final Bank	totals.final_bank	Bank + SSLZC received + Pathao received.
Owner	Cash/Bank Remain	owner.cash_after_cost / owner.bank_after_cost	Owner totals after owner-level costs.

5. Backend monthly pipeline

5.1 Store loading

The controller now loads all active stores plus inactive stores that have month activity in orders, payments, splits, refunds, branch costs, expense payments, or admin entries. This prevents old months from losing a branch column after the branch is deactivated.

5.2 Loader summary

Loader	Returns	Feeds columns
loadBranchSales	rawSales[store][date]	Branch Sale
loadBranchPayments	rawPayments[store][date][cash|bank]	Branch Cash/Bank before deductions
loadBranchExOn	branchExOn[store][date]	Ex/On
loadBranchCosts	branchCosts[store][date][total|cash|bank]	Daily Cost + deduction buckets
loadAdminEntries	adminData[store/_global][date][type]	Salary, cash-to-bank, SSLZC, Pathao
loadOnlineData	onlineData[date]	Online Sales/Advance/SSLZC/COD
loadOwnerEntries	ownerData[date][type]	Owner columns

5.3 Branch formula

```
raw_cash     = completed cash payments - completed cash refunds
raw_bank     = completed non-cash payments - completed non-cash refunds
sale         = branch order total on order date
ex_on        = exchange_surplus payments - exchange_refund refunds
daily_cost   = cash_cost + bank_cost
salary       = admin salary_setaside
cash_to_bank = admin cash_to_bank

displayed_cash = raw_cash - salary - cash_cost - cash_to_bank
```

Important change: displayed_cash is no longer max(0, ...). If raw cash is Tk 2,000 and deductions are Tk 5,500, the branch now shows -Tk 3,500 instead of hiding the shortage as 0.

5.4 Day and owner formulas

```
branch_total_cash = SUM(branch displayed_cash)
branch_total_bank = SUM(branch displayed_bank)
bank              = branch_total_bank + online.advance
final_bank        = bank + sslzc_received + pathao_received
total_sale        = SUM(branch daily_sale) + online.daily_sales

owner.total_cash      = branch_total_cash + owner.cash_invest
owner.total_bank      = final_bank + owner.bank_invest
owner.cash_after_cost = owner.total_cash - owner.cash_cost
owner.bank_after_cost = owner.total_bank - owner.bank_cost
```

6. Business date rules and yesterday/today fix

The repeated cash-sheet date issue was mostly caused by using processing/completion timestamps before the explicitly selected payment date. The updated controller now prefers payment_received_date for order payments. This means a payment intentionally recorded for yesterday stays yesterday, even if the user processed it today.

Bucket	Date source after fix
Branch sale	COALESCE(order_date, confirmed_at, created_at) -> business date
Branch normal payment	COALESCE(payment_received_date, completed_at, processed_at, created_at) -> business date
Branch split payment	COALESCE(op.payment_received_date, ps.completed_at, op.completed_at, ps.processed_at, op.processed_at, op.created_at) -> business date
Branch refund	refunds.completed_at -> business date
Manual branch cost	branch_cost_entries.entry_date as plain date
Accounting expense payment	COALESCE(expense_payments.completed_at, processed_at, expenses.expense_date) -> business date
Admin entry	admin_entries.entry_date as plain date
Online order sale/COD due	COALESCE(order_date, confirmed_at, created_at) -> business date
Online payment	COALESCE(payment_received_date, completed_at, processed_at, created_at) -> business date
Owner entry	owner_entries.entry_date as plain date

The datetime business-date conversion still defaults to Bangladesh/UTC +6 using CASH_SHEET_UTC_OFFSET_HOURS=6. If production timestamps are already saved in Bangladesh local time, set CASH_SHEET_UTC_OFFSET_HOURS=0 to prevent evening rows from jumping to the next day.

Event	Old behavior	New behavior
POS payment selected as 2026-07-04 but processed on 2026-07-05	Could hydrate 2026-07-05 because completed_at came first.	Hydrates 2026-07-04 because payment_received_date comes first.
No payment_received_date exists	Used completed/processed/created fallback.	Still uses completed/processed/created fallback.
UTC timestamp 2026-07-05 19:30	+6 made 2026-07-06.	Same unless env offset is changed.

7. Branch sales and payment buckets

7.1 Branch sale

```
Branch Sale = SUM(orders.total_amount)
where order_type in ['counter', 'pos', 'offline']
and order is not deleted
and status not in ['cancelled','canceled','refunded','void','deleted']
and metadata is not is_exchange_replacement
```

Sale is counted on order date. Payment is counted on payment date. Therefore a sale can appear on Jul 4 while cash/bank appears on Jul 5 if payment_received_date/completed_at is Jul 5.

7.2 Payments

Normal payments without split rows count the parent order_payments amount. Split payments count payment_splits only, preventing double counting parent + splits.

Payment example	Bucket	Effect
Tk 5,000 cash POS payment	raw_cash	+5,000
Tk 5,000 card/bKash/bank payment	raw_bank	+5,000
Parent payment Tk 10,000 with splits 4,000 cash + 6,000 bKash	split rows	raw_cash +4,000; raw_bank +6,000; parent ignored
payment_type = store_credit / exchange_balance / balance_carryover	excluded	No new money, so not counted

7.3 Refunds

Completed refunds subtract on refund completion date. Store credit and gift card refunds are excluded because no cash/bank leaves immediately. Null refund_method is now treated as a money refund instead of disappearing.

Refund example	Bucket	Effect
Cash refund Tk 1,500	raw_cash	-1,500
bKash/card/bank refund Tk 2,000	raw_bank	-2,000
store_credit refund Tk 2,000	none	No cash/bank movement
Legacy refund_method null Tk 1,000	raw_bank fallback	-1,000, so older rows are not ignored

8. Cancellations and refunds - corrected behavior

This is the most important business correction. Previously, a paid order could be cancelled/refunded. Because the payment query excluded parent order statuses like cancelled/refunded, the original payment disappeared. But the refund row still subtracted money. That created wrong negatives. The new rule separates commercial sale status from cash-flow history.

Case	Sale column	Cash/Bank column	Why
Unpaid order cancelled	Removed	No change	No payment/refund exists.
Paid order cancelled, no refund yet	Removed	Payment remains	Business still holds money until refund happens.
Paid order cancelled and refunded same day	Removed	Payment and refund net to 0 on that date	Money came in and went out.
Paid Jul 4, refunded Jul 6	Sale removed after cancellation/refund	Jul 4 shows money in; Jul 6 shows money out	Cash-flow is date accurate.
Order status refunded with completed refund	Sale excluded	Payment remains + refund subtracts	No double-negative.

Real example: POS order Tk 8,000 paid cash on Jul 4, then cancelled/refunded cash on Jul 6.

Date	Order status	Sale	Cash event	Branch cash effect
Jul 4 before cancellation	delivered/paid	+8,000	+8,000 payment	+8,000
Jul 4 after cancellation on Jul 6	cancelled/refunded	0	+8,000 payment still counted	+8,000
Jul 6 refund date	cancelled/refunded	0	-8,000 cash refund	-8,000
Month net		0 sale	+8,000 - 8,000	0

For same-day cancellation/refund, cash effect is +8,000 - 8,000 = 0. For different-day refund, the sheet correctly shows cash movement on both dates.

9. Return-exchange and Ex/On tracking

Exchange is treated as two layers: the actual cash/bank movement, and the Ex/On informational movement. Ex/On does not replace cash/bank; it explains why extra money was collected or refunded during exchange.

Exchange event	Source row	Cash/bank effect	Ex/On effect
Customer upgrades and pays extra Tk 600 cash	order_payments.payment_type = exchange_surplus	raw_cash +600	ex_on +600
Customer upgrades and pays extra Tk 600 bKash	exchange_surplus with non-cash method	raw_bank +600	ex_on +600
Customer downgrades and receives Tk 300 cash refund	refunds.refund_type = exchange_refund	raw_cash -300	ex_on -300
Customer receives store credit	refund_method store_credit	no cash/bank	depending refund row, Ex/On may show if exchange_refund

Example: Main Store has Tk 20,000 normal cash payment, exchange top-up Tk 600 cash, exchange refund Tk 300 cash. raw_cash = 20,000 + 600 - 300 = Tk 20,300. Ex/On = 600 - 300 = Tk 300.

10. Daily costs

Daily cost is the total branch operational cost for a date/store. It can come from manual cash-sheet Branch Costs or from completed ExpensePayment rows in the accounting expense module.

```
daily_cost     = cash_paid_cost + bank_paid_cost
displayed_cash = raw_cash - salary - cash_paid_cost - cash_to_bank
displayed_bank = raw_bank - bank_paid_cost + cash_to_bank
```

Source	Tables	Cash sheet effect	Double-count protection
Branch Cost panel	branch_cost_entries + mirrored expenses + expense_payments	daily_cost + amount; cash_cost + amount	Mirrored expense metadata source=cash_sheet_branch_cost is excluded from external expense loader.
Accounting cash expense	expenses + expense_payments + payment_methods.type=cash	daily_cost + amount; cash decreases	Only completed expense_payments and non-cancelled/rejected expenses count.
Accounting bKash/card/bank expense	payment_methods.type not cash	daily_cost + amount; bank decreases	Same completed/payment status checks.

Example: raw_cash Tk 14,100. Manual branch cost Tk 900 cash + accounting cash expense Tk 600 = cash_cost Tk 1,500. Accounting bKash expense Tk 700 = bank_cost Tk 700. daily_cost = 1,500 + 700 = Tk 2,200.

11. Admin entries

Admin entries are stored in admin_entries. Branch-scoped entries require store_id; global disbursement entries have store_id = null.

Type	Store required	Report effect	Ledger pair
salary_setaside	Yes	Salary column increases; cash decreases	Debit salary reserve, credit branch cash
cash_to_bank	Yes	Cash decreases; bank increases	Debit bank, credit branch cash
sslzc	No	SSLZC Received increases; final_bank increases	Debit bank, credit SSLCommerz receivable
pathao	No	Pathao Received increases; final_bank increases	Debit bank, credit Pathao receivable

Admin example

```
raw_cash     = Tk 14,100
raw_bank     = Tk 28,500
salary       = Tk 5,000
cash_cost    = Tk 1,500
bank_cost    = Tk 700
cash_to_bank = Tk 6,000

displayed_cash = 14,100 - 5,000 - 1,500 - 6,000 = Tk 1,600
displayed_bank = 28,500 - 700 + 6,000 = Tk 33,800
```

12. Online/ecommerce and disbursement logic

Online channels are separated because order value, customer payment, courier collection, and gateway/courier settlement can happen on different dates. The cash sheet must not mix visible receivable with actual bank deposit.

Bucket	How it is created	Added to bank/final bank?
online.daily_sales	SUM social_commerce/ecommerce order total on order date	No. It affects total sale only.
online.advance	Social-commerce non-COD payments minus social-commerce refunds	Yes. Added to day bank.
online.online_payment (SSLZC column)	Ecommerce non-COD gateway payments minus ecommerce refunds	No. Visible receivable until SSLZC received entry.
online.cod_due	Outstanding amount > 0 for COD/courier online orders	No. Receivable tracker.
online.cod_collected	COD/courier payment posted after delivery or auto-settled delivery metadata	No direct bank. It waits for Pathao/COD disbursement.
sslzc_received	Admin sslzc entry	Yes. Added to final_bank.
pathao_received	Admin pathao entry	Yes. Added to final_bank.

Online example

```
Social prepaid order      = Tk 2,500 -> online.advance +2,500 -> bank +2,500
Ecommerce SSLCommerz paid = Tk 3,200 -> online.online_payment +3,200 -> final_bank unchanged
COD order outstanding     = Tk 4,000 -> online.cod_due +4,000 -> bank unchanged
Admin SSLZC received      = Tk 3,000 -> final_bank +3,000
Admin Pathao received     = Tk 3,800 -> final_bank +3,800
```

13. Owner entries and final after-cost totals

Owner entries do not change branch sale, branch cash, branch bank, online buckets, or disbursements. They sit after day totals and show owner-level injections or owner-level costs.

Owner type	Effect
cash_invest	owner.total_cash = totals.cash + cash_invest
bank_invest	owner.total_bank = totals.final_bank + bank_invest
cash_cost	owner.cash_after_cost = owner.total_cash - cash_cost
bank_cost	owner.bank_after_cost = owner.total_bank - bank_cost

```
Day totals: cash = Tk 6,600; final_bank = Tk 54,600
Owner cash investment = Tk 20,000
Owner bank investment = Tk 50,000
Owner cash cost = Tk 2,000
Owner bank cost = Tk 3,000

owner.total_cash      = 6,600 + 20,000 = Tk 26,600
owner.total_bank      = 54,600 + 50,000 = Tk 104,600
owner.cash_after_cost = 26,600 - 2,000 = Tk 24,600
owner.bank_after_cost = 104,600 - 3,000 = Tk 101,600
```

14. Full real-value day simulation

Assume one business date: 2026-07-05. There are two stores: Main Store and Branch X. These values are chosen to force every important bucket to update.

#	Event/source	Amount
1	Main Store POS/offline sales	Tk 38,000
2	Branch X POS/offline sales	Tk 20,000
3	Main cash payments	Tk 15,000
4	Main bank/card/bKash payments	Tk 30,500
5	Main cash refund	-Tk 1,200
6	Main bank refund	-Tk 2,000
7	Main exchange surplus cash	+Tk 600
8	Main exchange refund cash	-Tk 300
9	Main branch costs cash	Tk 1,500
10	Main branch costs bank	Tk 700
11	Main salary set-aside	Tk 5,000
12	Main cash-to-bank	Tk 6,000
13	Branch X cash payments	Tk 8,500
14	Branch X bank payments	Tk 11,500
15	Branch X costs cash	Tk 1,000
16	Branch X salary	Tk 2,500
17	Social prepaid advance	Tk 2,500
18	Ecommerce SSLCommerz visible payment	Tk 3,200
19	COD due	Tk 4,000
20	SSLZC received	Tk 3,000
21	Pathao received	Tk 3,800
22	Owner cash/bank invest	Tk 20,000 / Tk 50,000
23	Owner cash/bank costs	Tk 2,000 / Tk 3,000

14.1 Branch raw bucket calculation

Store	Raw sale	Raw cash	Raw bank	Ex/On
Main Store	38,000	15,000 + 600 - 1,200 - 300 = 14,100	30,500 - 2,000 = 28,500	600 - 300 = 300
Branch X	20,000	8,500	11,500	0

14.2 Branch displayed cash/bank

Store	Cost	Salary	Cash-to-bank	Displayed cash	Displayed bank
Main Store	cash 1,500 / bank 700	5,000	6,000	14,100 - 5,000 - 1,500 - 6,000 = 1,600	28,500 - 700 + 6,000 = 33,800
Branch X	cash 1,000 / bank 0	2,500	0	8,500 - 2,500 - 1,000 = 5,000	11,500

14.3 Online and final totals

```
online.daily_sales = 2,500 + 3,200 + 4,000 = Tk 9,700
online.advance     = Tk 2,500
online.SSLZC       = Tk 3,200 (visible, not final bank)
online.COD/Due     = Tk 4,000

branch_total_sale = 38,000 + 20,000 = Tk 58,000
branch_cash       = 1,600 + 5,000 = Tk 6,600
branch_bank       = 33,800 + 11,500 = Tk 45,300

total_sale = 58,000 + 9,700 = Tk 67,700
bank       = 45,300 + 2,500 = Tk 47,800
final_bank = 47,800 + 3,000 + 3,800 = Tk 54,600
```

14.4 Final row

Section	Displayed values
Main Store	Sale 38,000 | Cash 1,600 | Bank 33,800 | Ex/On 300 | Salary 5,000 | Cost 2,200 | ->Bank 6,000
Branch X	Sale 20,000 | Cash 5,000 | Bank 11,500 | Salary 2,500 | Cost 1,000
Online	Sales 9,700 | Advance 2,500 | SSLZC 3,200 | COD/Due 4,000
Disbursement	SSLZC Received 3,000 | Pathao Received 3,800
Day totals	Total Sale 67,700 | Cash 6,600 | Bank 47,800 | Final Bank 54,600
Owner	+Cash 20,000 | Total Cash 26,600 | Cash-Cost 2,000 | +Bank 50,000 | Total Bank 104,600 | Bank-Cost 3,000 | Cash Remain 24,600 | Bank Remain 101,600

15. Scenario-by-scenario simulations

Scenario	Setup	Expected sheet result
A. Branch order created, no payment	Order total Tk 8,000, order_type counter.	Sale +8,000. Cash/Bank unchanged.
B. Cash payment completed	Completed cash payment Tk 8,000 with payment_received_date Jul 5.	raw_cash +8,000 on Jul 5.
C. Bank/bKash/card payment completed	Completed non-cash payment Tk 8,000.	raw_bank +8,000.
D. Split payment	Parent Tk 10,000; splits Tk 4,000 cash + Tk 6,000 bKash.	raw_cash +4,000; raw_bank +6,000; no parent double count.
E. Payment backdated	Processed today but payment_received_date is yesterday.	Cash/bank hydrates yesterday.
F. Branch cost panel	Manual branch cost Tk 900.	daily_cost +900; cash decreases 900; mirrored expense excluded from double count.
G. External bKash expense	ExpensePayment Tk 700 with non-cash payment method.	daily_cost +700; bank decreases 700.
H. Salary set-aside	salary_setaside Tk 3,000.	Salary +3,000; cash decreases 3,000.
I. Cash-to-bank	cash_to_bank Tk 10,000.	Cash -10,000; bank +10,000.
J. Social advance	Social-commerce bKash payment Tk 2,500.	online.advance +2,500; day bank +2,500.
K. Ecommerce gateway	Ecommerce SSLCommerz payment Tk 3,200.	online.SSLZC +3,200; final bank unchanged until settlement.
L. SSLZC received	Admin sslzc Tk 3,000.	final_bank +3,000.
M. COD due	COD/courier order outstanding Tk 4,000.	COD/Due +4,000; bank unchanged.
N. Pathao received	Admin pathao Tk 3,800.	final_bank +3,800.
O. Paid order cancelled before refund	Order sale removed by cancelled status, payment remains completed.	Sale 0; cash/bank remains until refund row exists.
P. Paid order refunded	Refund completed Tk 8,000.	Refund date cash/bank -8,000; payment date remains +8,000.
Q. Exchange upgrade	exchange_surplus Tk 600 cash.	raw_cash +600; Ex/On +600.
R. Exchange downgrade	exchange_refund Tk 300 cash.	raw_cash -300; Ex/On -300.
S. Cash deficit	raw_cash 2,000, salary 3,000, cost 1,000.	cash = 2,000 - 3,000 - 1,000 = -2,000; visible in red.
T. Inactive branch history	Branch inactive now but had Jul 5 payments/costs.	Branch still appears in July sheet.

16. QA test plan

Run these tests after deploying the patched backend and frontend. The expected values are intentionally simple so QA can calculate by hand.

Test	Setup	Expected result
Route consolidation	Open /cash-sheet and /cash-sheet-new	/cash-sheet loads sheet; /cash-sheet-new redirects to /cash-sheet. API /cash-sheet-new should not be used.
Backdated payment	Create POS payment Tk 5,000 with payment_received_date yesterday	Yesterday cash/bank +5,000; today unchanged.
Same-day paid cancellation/refund	Create cash order 8,000, cancel and refund same day	Sale removed; cash net 0 for the day.
Different-day refund	Pay 8,000 on Jul 4, refund Jul 6	Jul 4 cash +8,000; Jul 6 cash -8,000; sale removed if order cancelled/refunded.
Split payment	10,000 split 4,000 cash + 6,000 bKash	Cash +4,000; bank +6,000 only.
Negative cash	raw_cash 2,000; salary 3,000; branch cash cost 1,000	Cash displays -2,000 in red, not 0.
Branch cost double count	Add branch cost 900	daily_cost +900 only once despite mirrored expense.
External expense by bKash	Expense payment 700 bKash	daily_cost +700; bank -700.
COD filter	Create non-COD online partial due and COD due	Only COD/courier due appears in COD/Due.
SSLZC visible vs received	Ecommerce payment 3,200, no admin sslzc	SSLZC visible +3,200; final_bank unchanged.
SSLZC disbursement	Admin sslzc 3,000	final_bank +3,000 on entry date.
Inactive branch	Deactivate branch after old month has activity	Old month still shows branch column and totals.
Exchange upgrade/downgrade	Top-up 600 and exchange refund 300	Ex/On net +300; cash/bank also reflect movements.

17. Debugging query templates

Use the same date source priority as the code. Replace the date range and store ID as needed. These queries are examples for MySQL-style production DB.

17.1 Branch sale

```
SELECT store_id,
DATE(DATE_ADD(COALESCE(order_date, confirmed_at, created_at), INTERVAL 6 HOUR)) AS day,
SUM(total_amount) AS sale
FROM orders
WHERE order_type IN ('counter','pos','offline')
AND deleted_at IS NULL
AND LOWER(status) NOT IN ('cancelled','canceled','refunded','void','deleted')
GROUP BY store_id, day;
```

17.2 Branch normal payment

```
SELECT op.store_id,
DATE(DATE_ADD(COALESCE(op.payment_received_date, op.completed_at, op.processed_at, op.created_at), INTERVAL 6 HOUR)) AS d
pm.type,
SUM(op.amount) AS amount
FROM order_payments op
JOIN payment_methods pm ON pm.id = op.payment_method_id
JOIN orders o ON o.id = op.order_id
LEFT JOIN payment_splits ps_probe ON ps_probe.order_payment_id = op.id
WHERE ps_probe.id IS NULL
AND op.status IN ('completed','partially_refunded','refunded')
AND op.deleted_at IS NULL
GROUP BY op.store_id, day, pm.type;
```

17.3 Split payment

```
SELECT COALESCE(ps.store_id, op.store_id, o.store_id) AS store_id,
DATE(DATE_ADD(COALESCE(op.payment_received_date, ps.completed_at, op.completed_at, ps.processed_at, op.processed_at, op.c
pm.type,
SUM(ps.amount) AS amount
FROM payment_splits ps
JOIN order_payments op ON op.id = ps.order_payment_id
JOIN orders o ON o.id = op.order_id
JOIN payment_methods pm ON pm.id = ps.payment_method_id
WHERE op.status IN ('completed','partially_refunded','refunded')
GROUP BY store_id, day, pm.type;
```

17.4 Refunds

```
SELECT COALESCE(pr.received_at_store_id, pr.store_id, o.store_id) AS store_id,
DATE(DATE_ADD(r.completed_at, INTERVAL 6 HOUR)) AS day,
r.refund_method,
SUM(r.refund_amount) AS amount
FROM refunds r
LEFT JOIN product_returns pr ON pr.id = r.return_id
JOIN orders o ON o.id = r.order_id
WHERE r.status = 'completed'
AND r.completed_at IS NOT NULL
AND (r.refund_method IS NULL OR r.refund_method NOT IN ('store_credit','gift_card'))
GROUP BY store_id, day, r.refund_method;
```

18. Developer handoff checklist

- Use /api/cash-sheet only. Do not add new logic under /api/cash-sheet-new.
- Use services/cashSheetService.ts for new frontend cash-sheet work.
- Do not calculate cash sheet totals in the frontend. The backend owns formulas.
- When fixing date bugs, inspect payment_received_date before completed_at.
- When fixing cancellation/refund bugs, separate commercial sale status from real cash-flow rows.
- When fixing return/exchange bugs, check both Ex/On and actual cash/bank movement.
- When fixing cost bugs, compare branch_cost_entries and external expense_payments to avoid double count.
- When branch cash looks 0, verify it is not actually negative. Negative cash is now displayed.
- When old months lose branches, check loadCashSheetStores and inactive stores with source activity.
- After any patch, run the QA matrix with hand calculations before trusting screenshots.

Final mental model

The corrected cash sheet is now a single canonical live aggregation system. Sales answer 'what valid order value exists'. Cash and bank answer 'what real money moved'. Admin and owner panels answer 'what finance/admin adjustments were made'. Online buckets answer 'what is paid, receivable, or settled'. When a number is wrong, debug the exact source bucket, not the whole page.